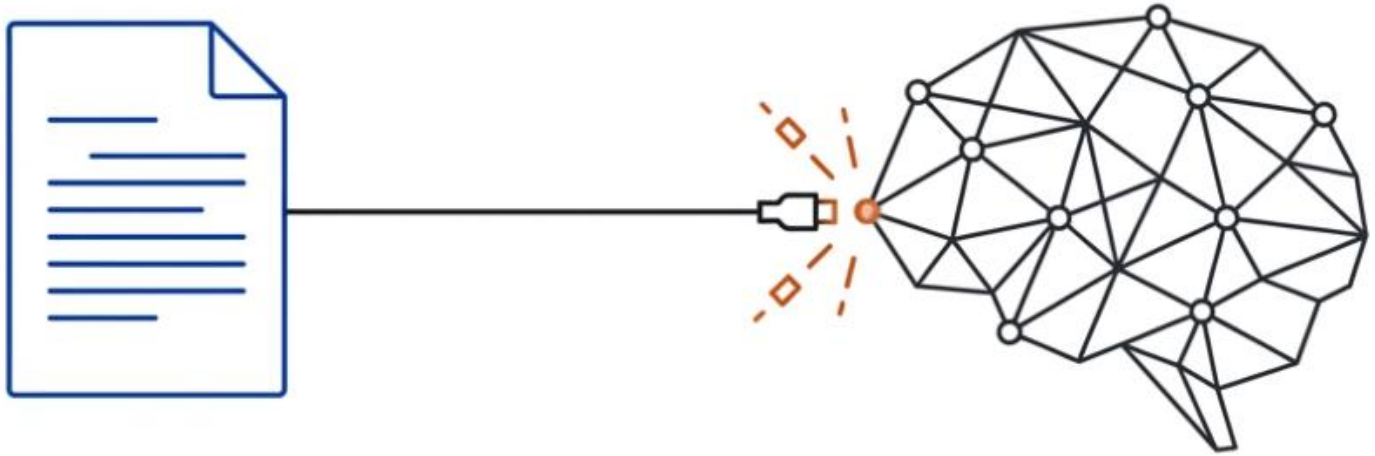
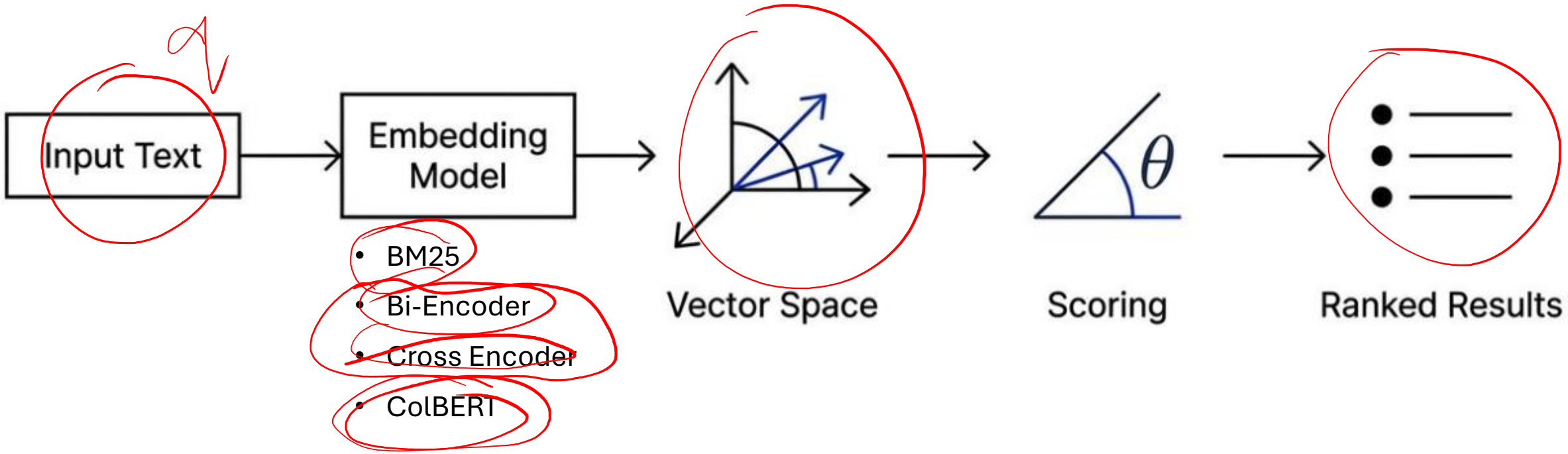


Storing & Searching Efficiently

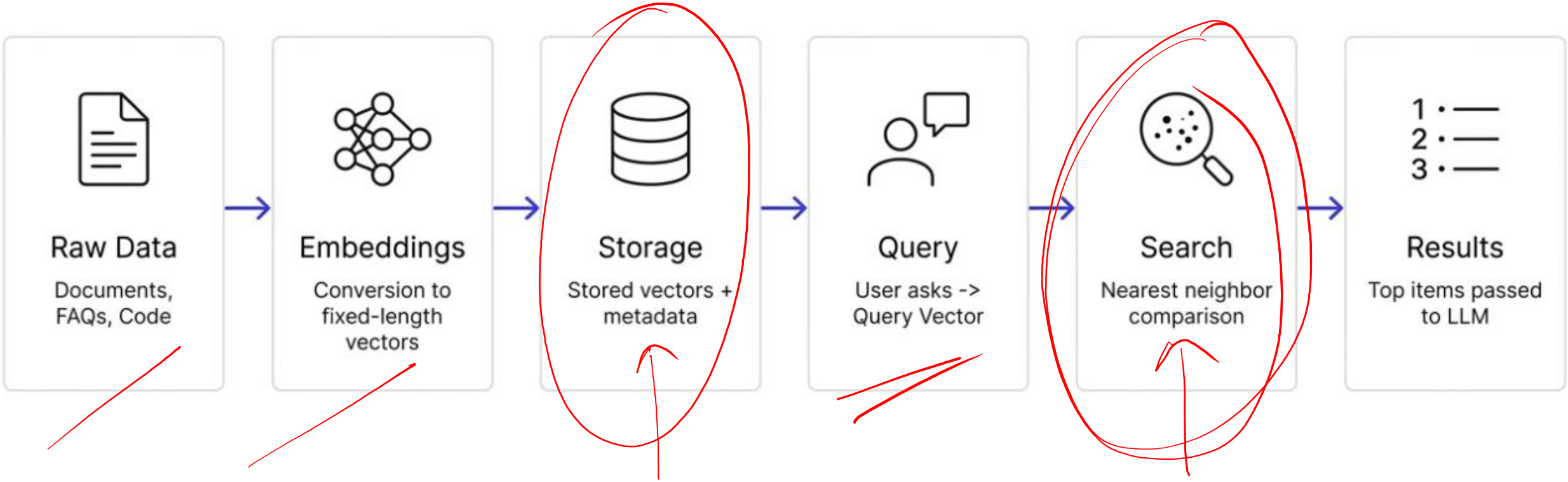
Vector DB + ANN + Indexing Choices. Vector DB. FAISS.
Speed/accuracy trade-offs and indexing impacts.

https://tinyurl.com/rag-augmented

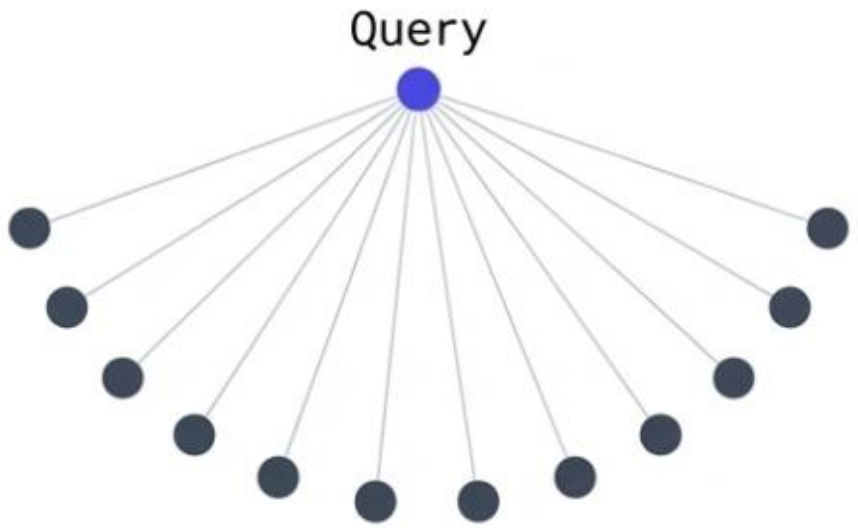
Previously...


Input Text
Embedding Model
Vector Space
Scoring
Ranked Results
• BM25
• Bi-Encoder
• Cross Encoder
• ColBERT

Scaling Up Pipeline

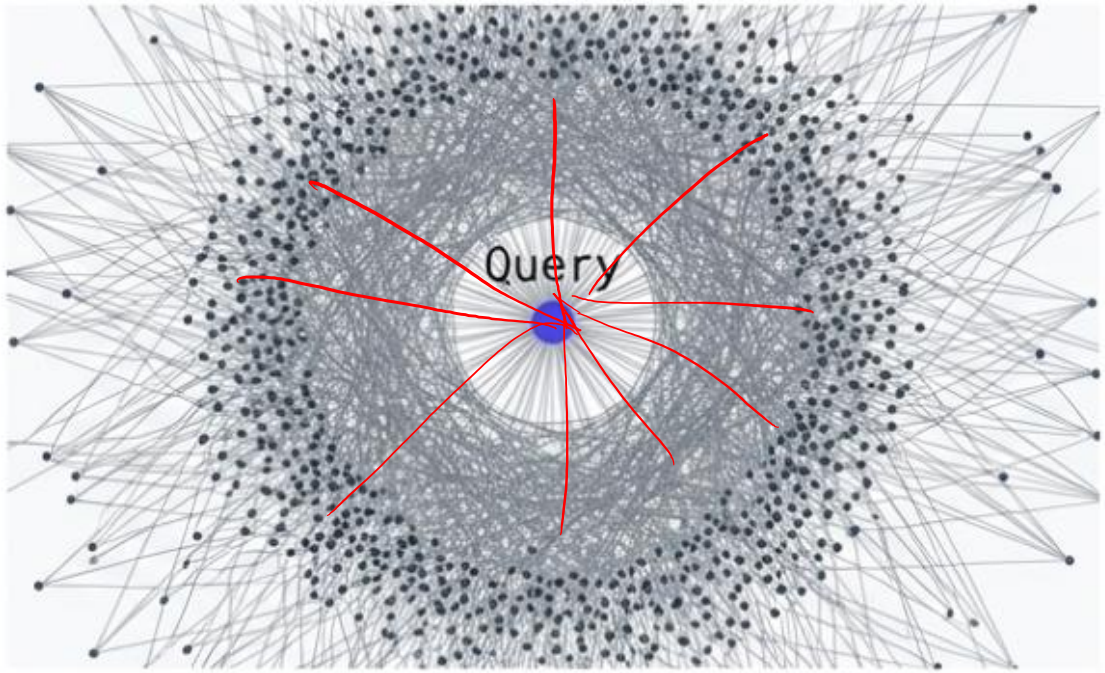


When exact search breaks down



Fast & Exact.

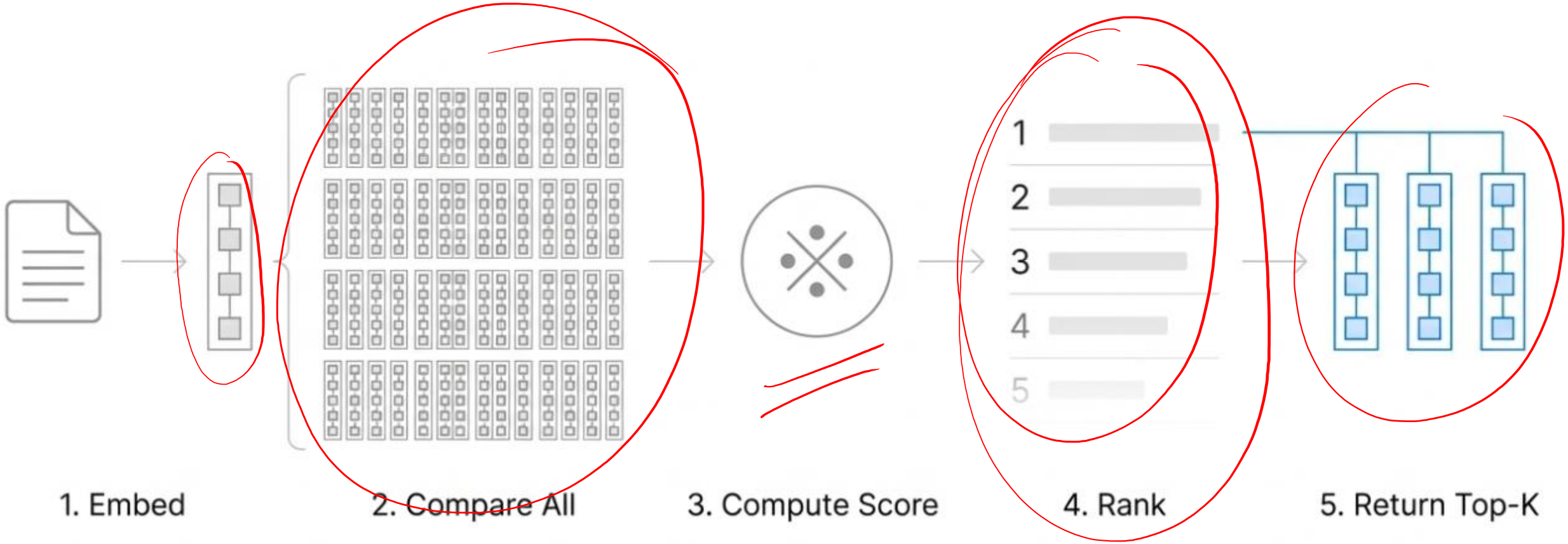
Exhaustive comparison (e.g., `cosine similarity`, `negative L2`) works **perfectly** for <20k vectors.



The Latency Bottleneck.

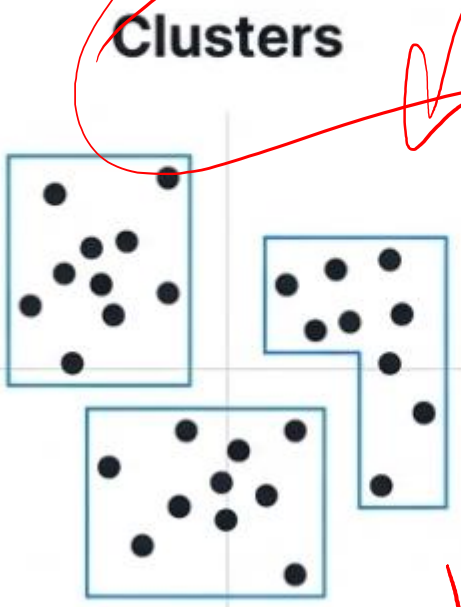
At 100M+ vectors, memory bandwidth and **latency** make real-time **exact search** impossible. Throughput drops; serving costs rise.

Exact match evaluates every candidate (KNN)

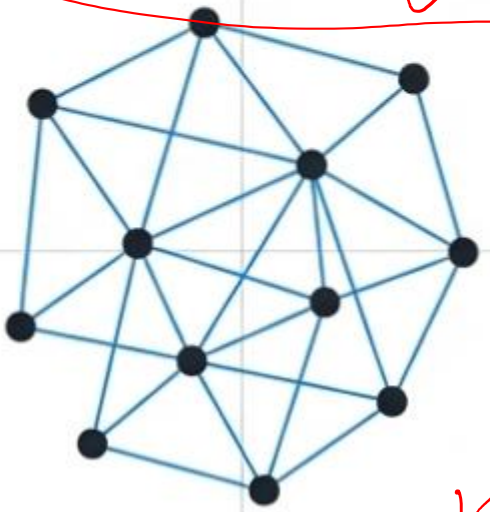


Approximate Nearest Neighbour (ANN)

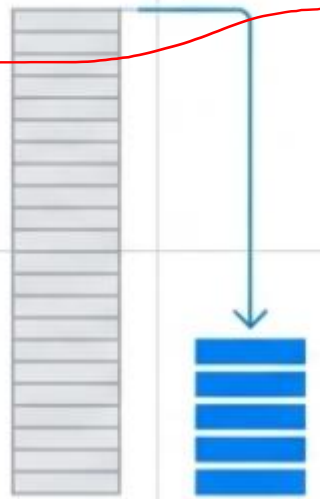
ANN is not random guessing. It uses specialized data structures and search strategies to avoid exhaustive comparison. By clustering, graphing, or compressing data, the system strictly evaluates only the most promising vector candidates.



IVF



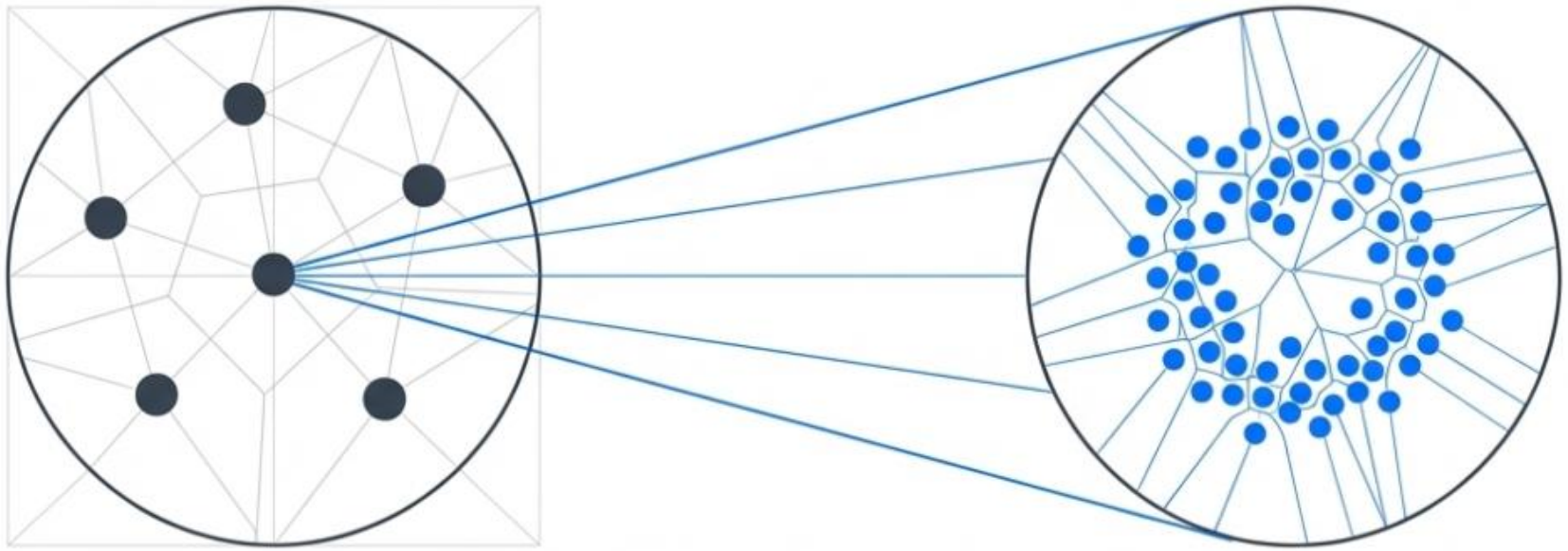
HNSW



PQ

Inverted File Index (IVF)

Do not search the entire database. Narrow the search to promising regions first, then search only inside those regions.



1. Coarse Search

Search among cluster centers to identify relevant regions.

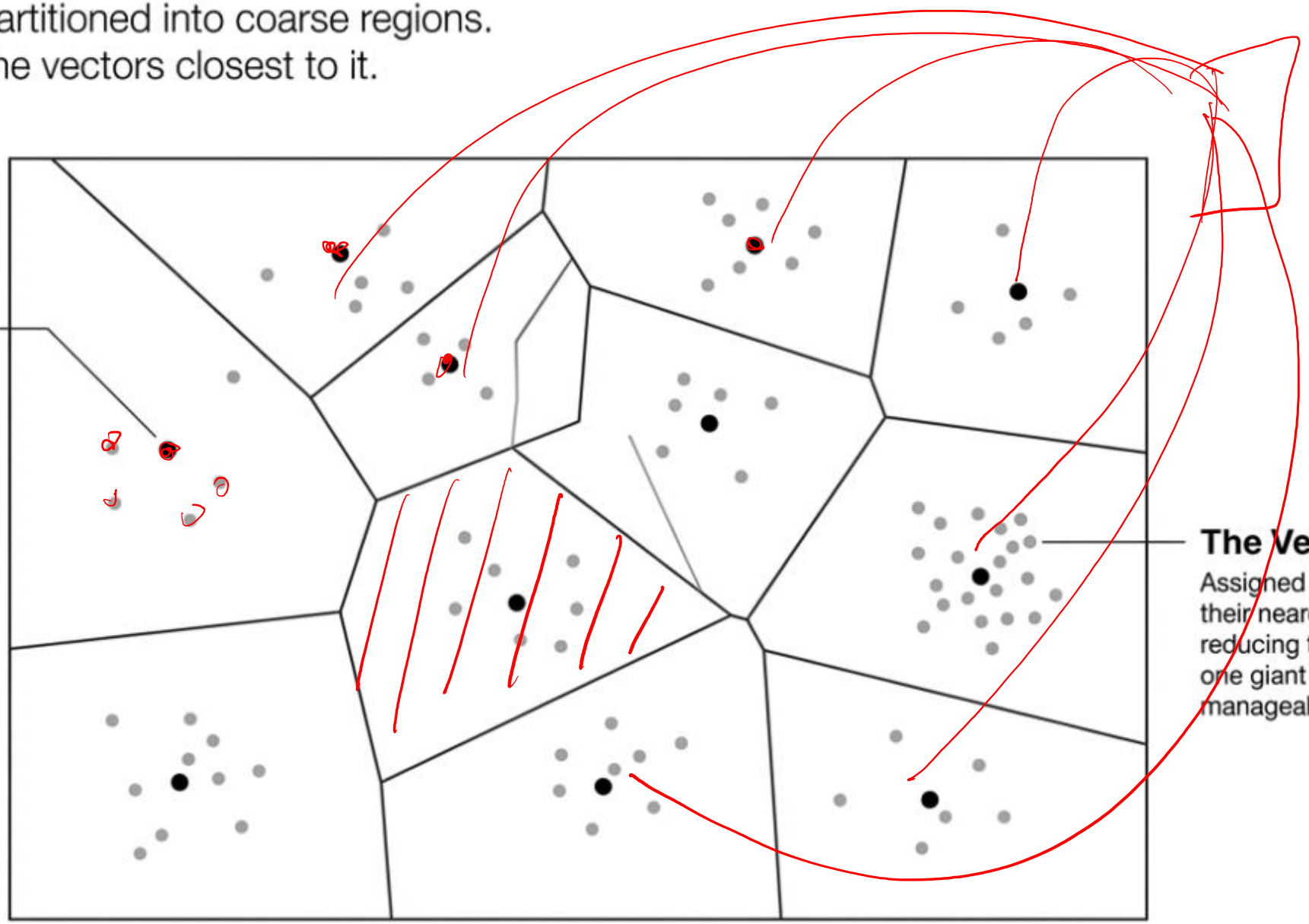
2. Fine Search

Search among actual data vectors only inside those selected regions.

The vector space is partitioned into coarse regions.
Each centroid owns the vectors closest to it.

The Centroid

A compact summary of the space. It gives a fast, coarse coarse hint about where in the space an answer might lie.

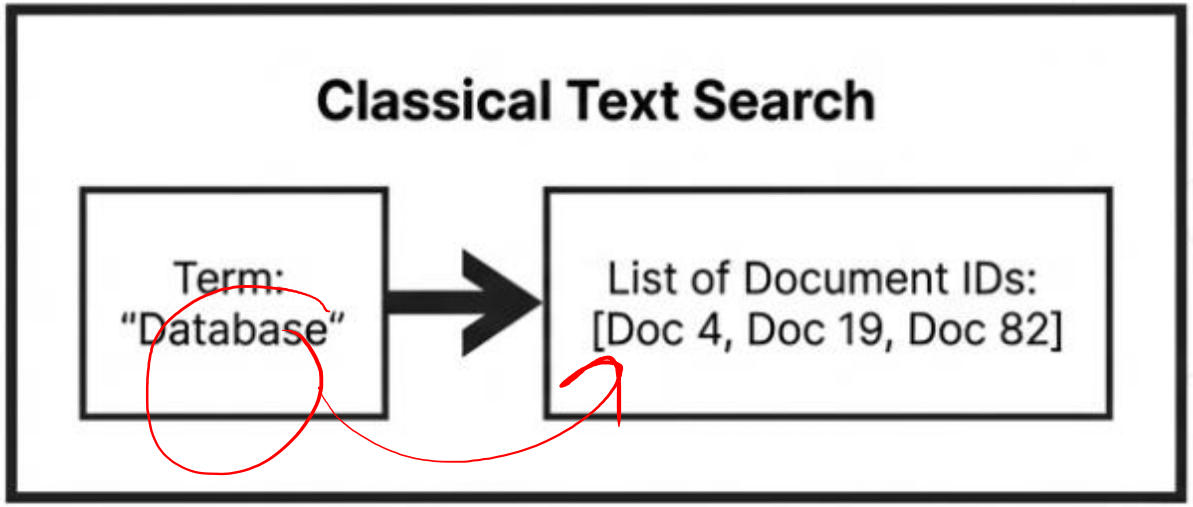


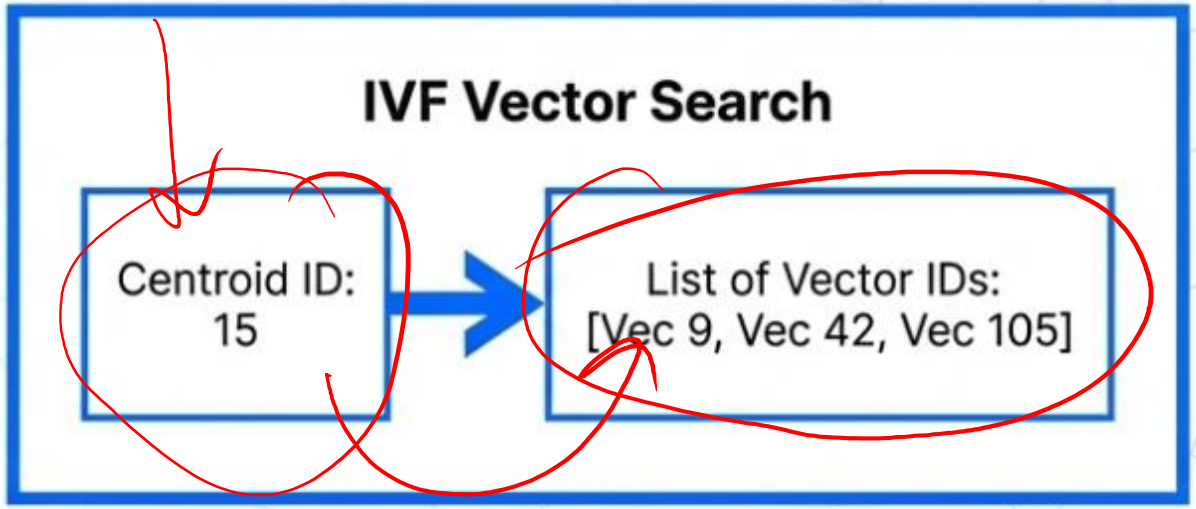
The Vectors

Assigned to the region of their nearest centroid, reducing the database from one giant pool into separate, manageable neighborhoods.

Why Inverted File?

The name is borrowed from classical information retrieval. Instead of a term pointing to documents, a cluster center acts as the entry point, pointing to a list of vectors assigned to that cluster.





Unlike Flat search, IVF cannot be used immediately on arbitrary vectors. It requires a dedicated training and building phase to learn the centroids and structure the lists before any query can be routed.

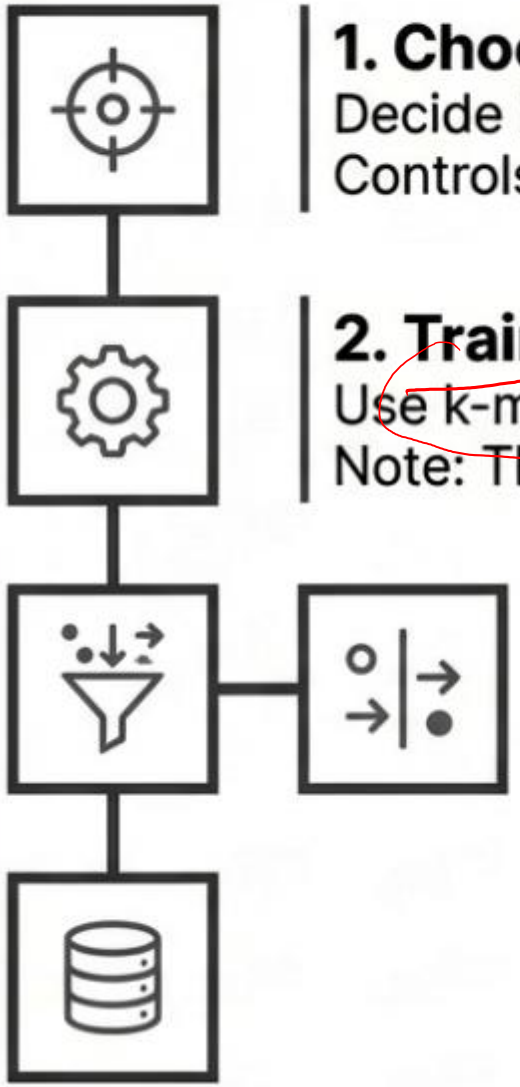
Phase A: Build Index (Offline)

Happens before queries are served.
Requires training.

Phase B: Search Index (Online)

Happens at query time.
Fast and approximate.

Phase A: Building the Index



1. Choose nlist

Decide how many regions to divide the space into (e.g., 100, 1,000, 10,000).
Controls partitioning granularity.

2. Train the Coarse Quantizer

Use k-means clustering on a representative sample to learn cluster centers.
Note: These centroids are learned representative points, not actual documents.

3. Assign Vectors

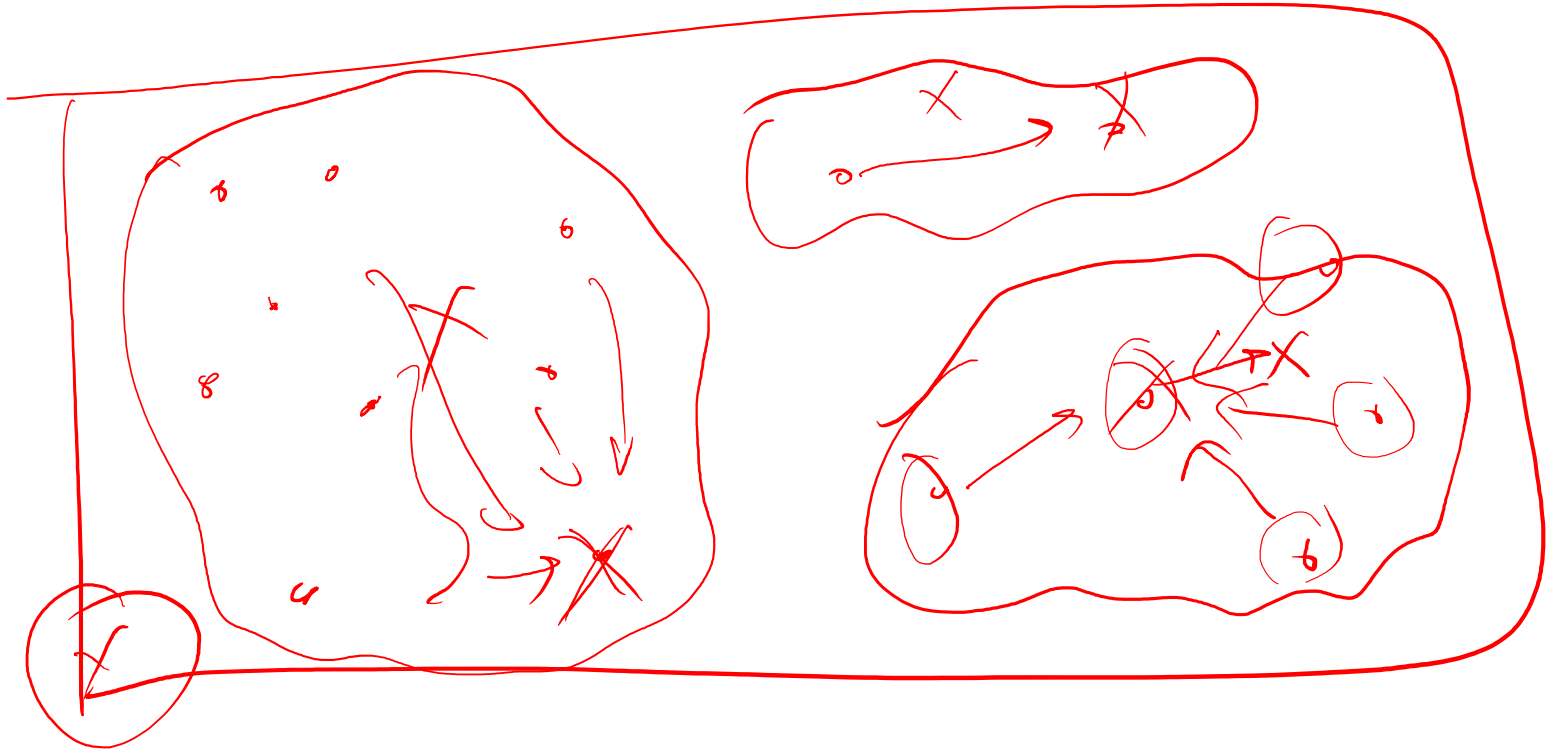
Compute distance from every stored vector to all centroids, assigning each to its nearest centroid.

4. Store the Lists

The database is now organized as separate posting lists attached to learned centroids, rather than one giant pool.

K-Means Clustering

3



Phase B: Query-Time Centroid Routing

Step 1: Coarse Search

The query is scored only against the cluster centroids, not the data vectors. Comparing against 1,000 centroids is vastly cheaper than millions of documents.

Step 2: Select Top Clusters

The system picks a small number of best centroids, controlled by the parameter nprobe (e.g., 1, 5, or 10 closest clusters).

Step 3: Gather Candidates

IVF collects all vectors stored exclusively in those selected inverted lists. This subset becomes the candidate set.

The Math of Efficiency: A Concrete Example

1,000,000 Operations
(Exact Search)

7,000 Operations
(IVF Search)
[1,000 Centroids] +
[6,000 Candidates]

The Setup:

- Database of 1,000,000 vectors.
- nlist = 1000 (1000 trained centroids)
- nprobe = 5 (search the top 5 clusters)

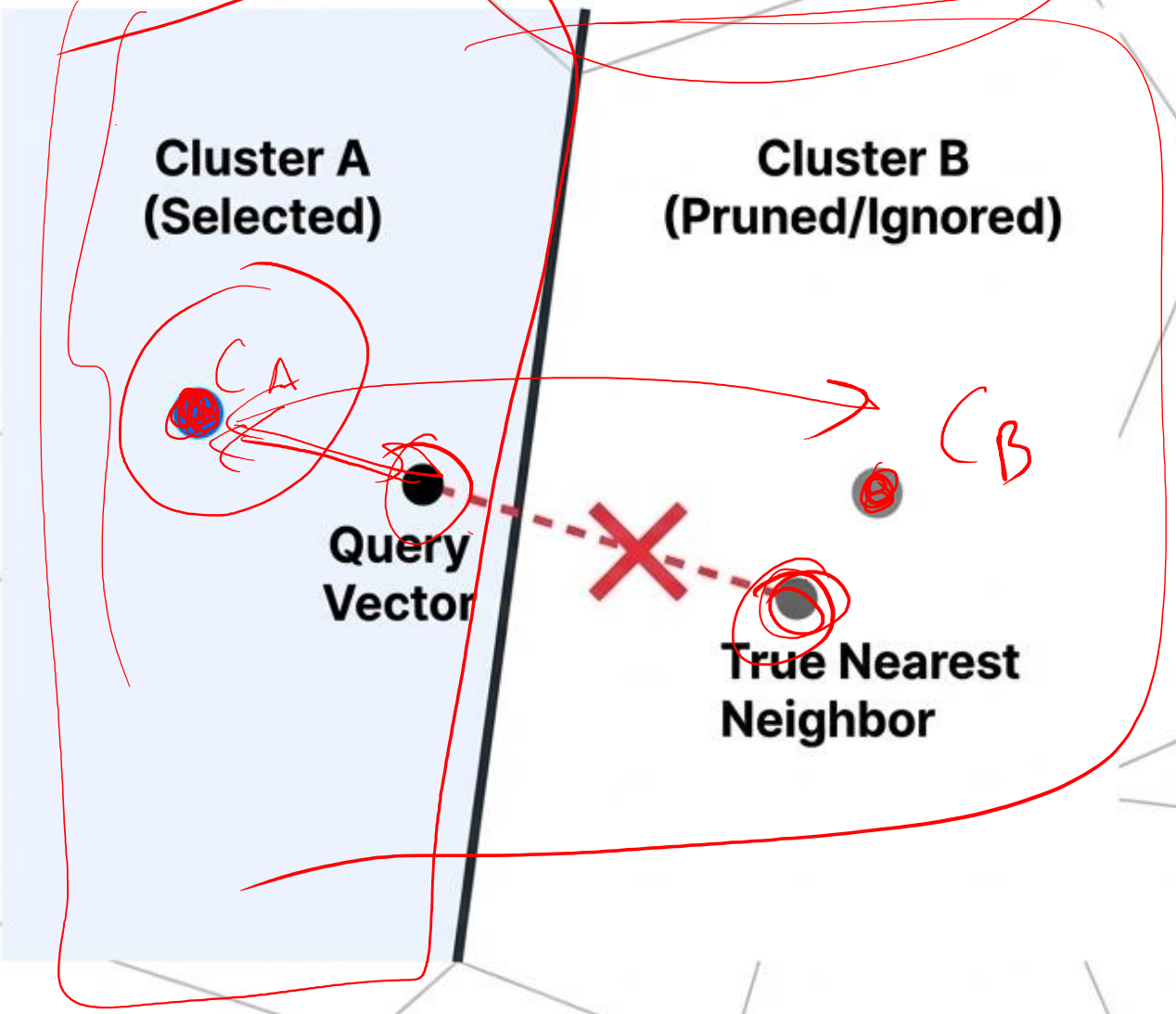
The Execution:

1. Query is compared to 1000 centroids.
2. Top 5 centroids are selected.
3. Vectors from those 5 clusters are gathered (~6,000 vectors total).
4. Query is scored against only those 6,000 vectors.

The Result:

IVF requires only **7,000** comparisons vs. 1,000,000 for exact search—a **massive structural reduction** in work.

The Source of Approximation: Cluster Pruning



Key Concept

IVF is approximate not because its fine scoring is wrong, but because it may never search the right cluster.

The Mechanism

If the true nearest neighbor belongs to a cluster not selected among the top nprobe centroids, it will never be scored.

The Distinction

- Inside searched clusters: IVF scores exactly.
- Globally: IVF is approximate because it ignores unselected clusters.

System Controls: Tuning the Index

nlist (Granularity)

Number of clusters in the coarse partition. Shapes the index structure.

- Small nlist: Large candidate lists, expensive fine search.
- Large nlist: Smaller candidate sets, faster search, but relevant neighbors may spread across more clusters.

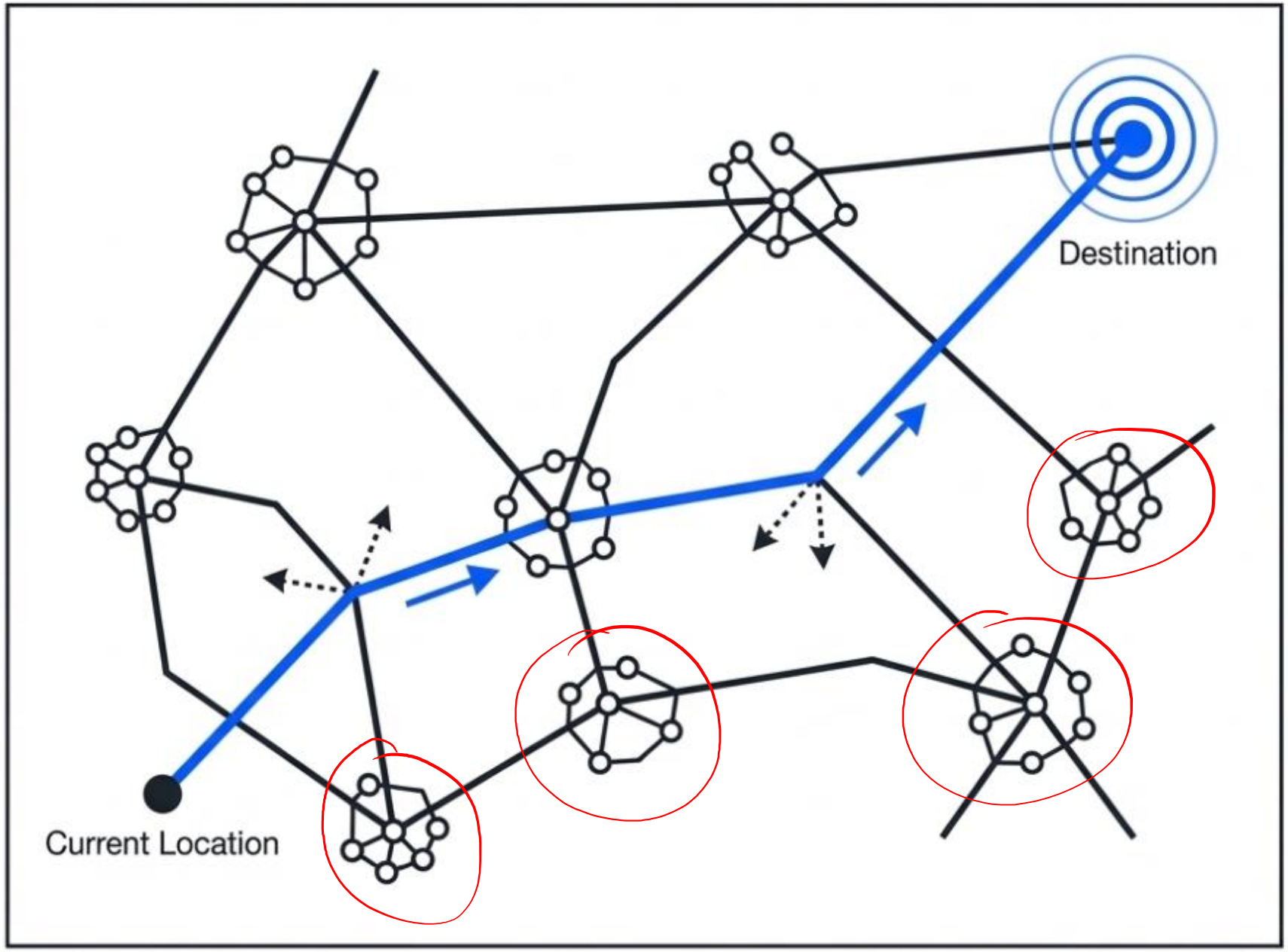
nprobe (Search Breadth)

Number of clusters searched at query time. The most direct knob for the speed vs. recall trade-off.

- Low nprobe: Very fast, but higher risk of missing true neighbors.
- High nprobe: Slower, handles cluster edge-cases better, resulting in higher recall.

Vector search as a city navigation problem

HNSW



The Flat Approach

Checking every building in the city to see which is closest to the destination.

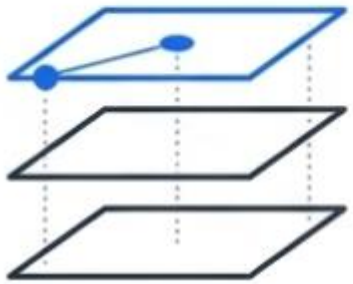
The HNSW Approach

Following a network of connected roads, consistently moving toward neighborhoods that appear closer to the target until you arrive.

The Mechanism

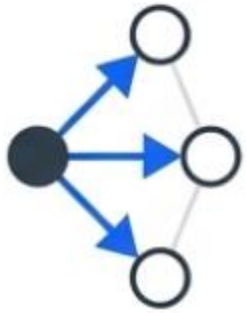
This is a greedy search. Compare the query with a node, look at its neighbors, and move to the neighbor if it is closer.

Unpacking the HNSW architecture



Hierarchical

Multiple graph layers. Higher layers are sparse for long jumps; lower layers are dense for precision.



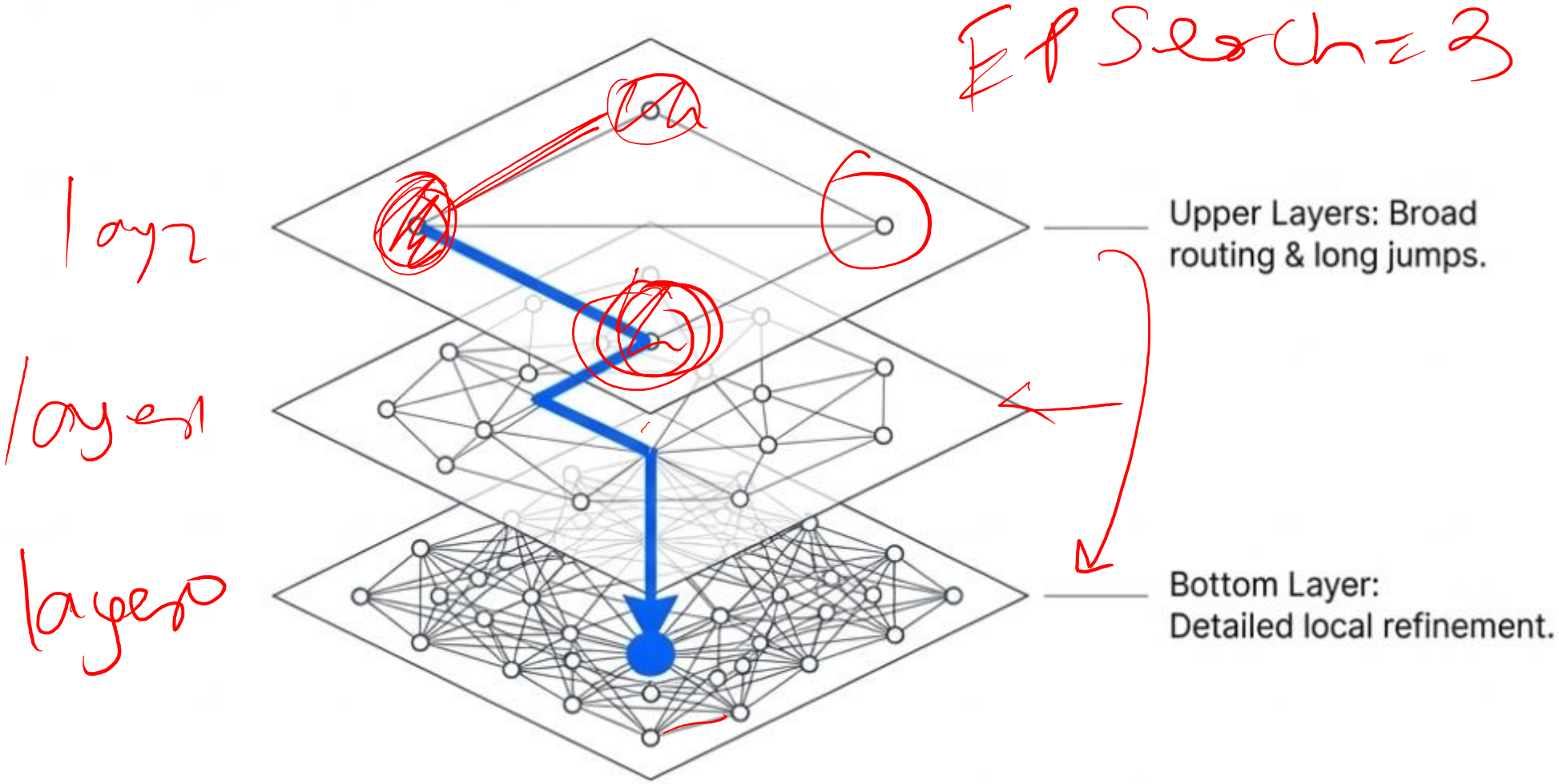
Navigable

The graph structure ensures that traversing node-to-node consistently leads toward valid nearest neighbor candidates.

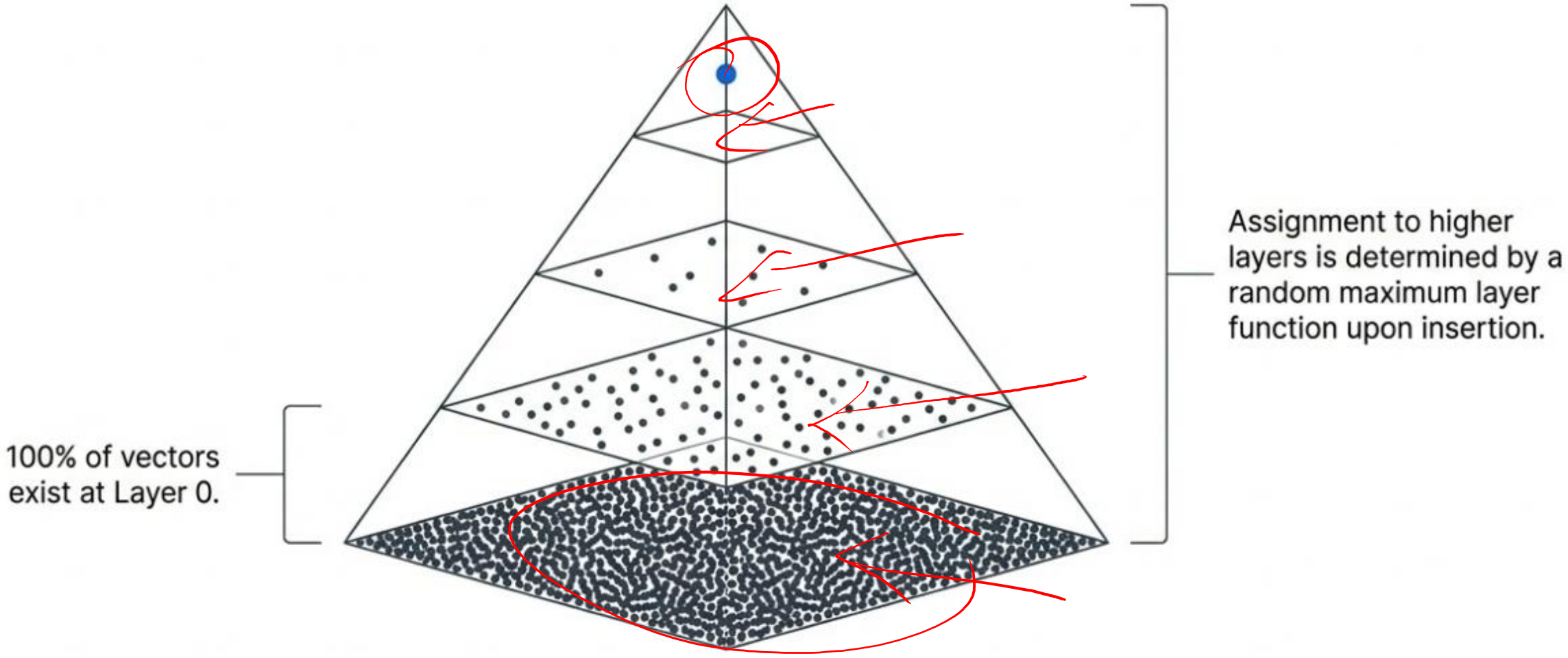
Small World

Built with a mix of dense local connections and long-range shortcuts, allowing any two nodes to be reached in very few steps.

Top layers help global routing. Bottom layers help local precision. HNSW allows search algorithms to jump across vast vector spaces instantly before zooming in on the exact neighborhood.



Building the index via a probabilistic pyramid

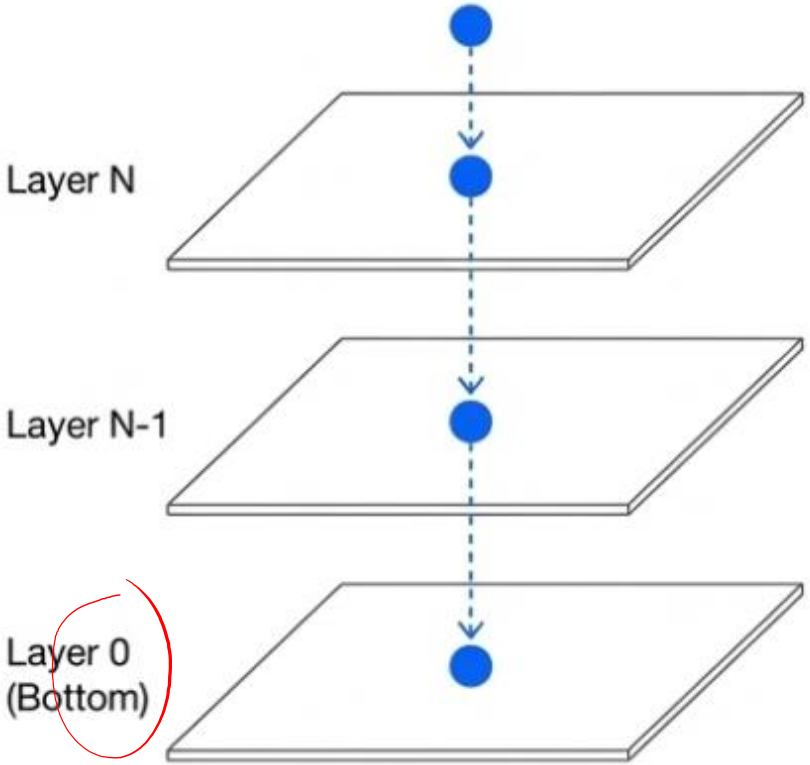


Unlike IVF, HNSW does not cluster the full dataset at once. It inserts vectors incrementally. Each new vector is randomly assigned a maximum highest layer. This mathematically guarantees a wide base of dense local connectivity and a tiny peak of long-range shortcuts.

Searching the index from coarse to fine

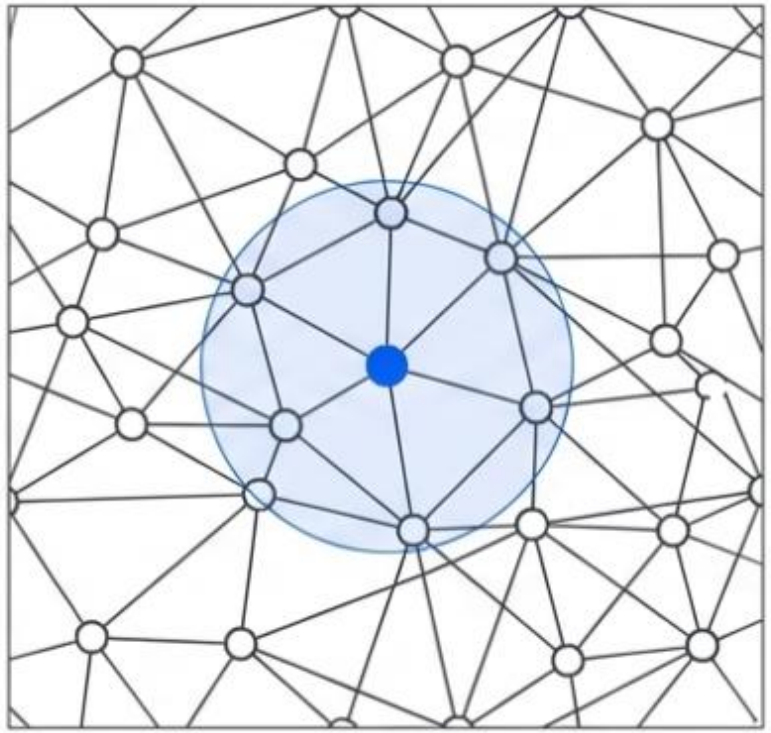
At upper layers, HNSW search is strictly greedy and focused on rapid routing. At the bottom layer, the search expands into a controlled local exploration to collect the strongest candidate nearest neighbors.

The Descent



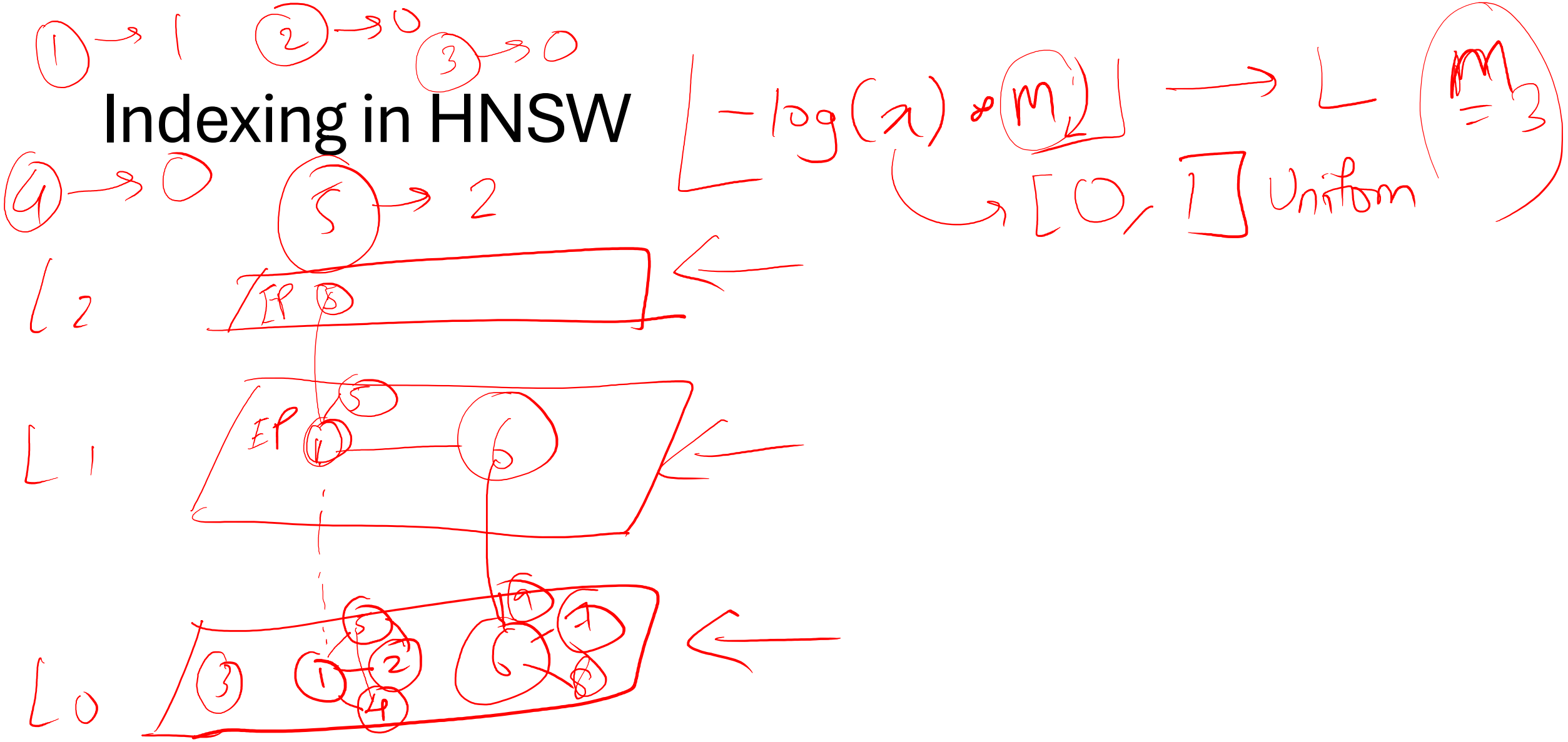
- 1. Embed Query
- 2. Enter Top Layer
- 3. Greedy Descend

The Floor Search



- 4. Broad Exploration
- 5. Return Top-K

Indexing in HNSW

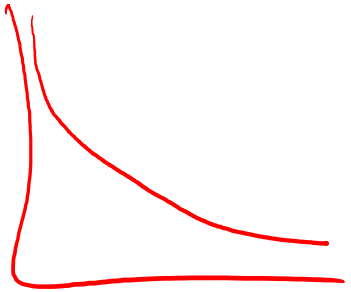


$$\boxed{m_L = \frac{1}{m}}$$

$m_L \ll 1$

~ 0.1

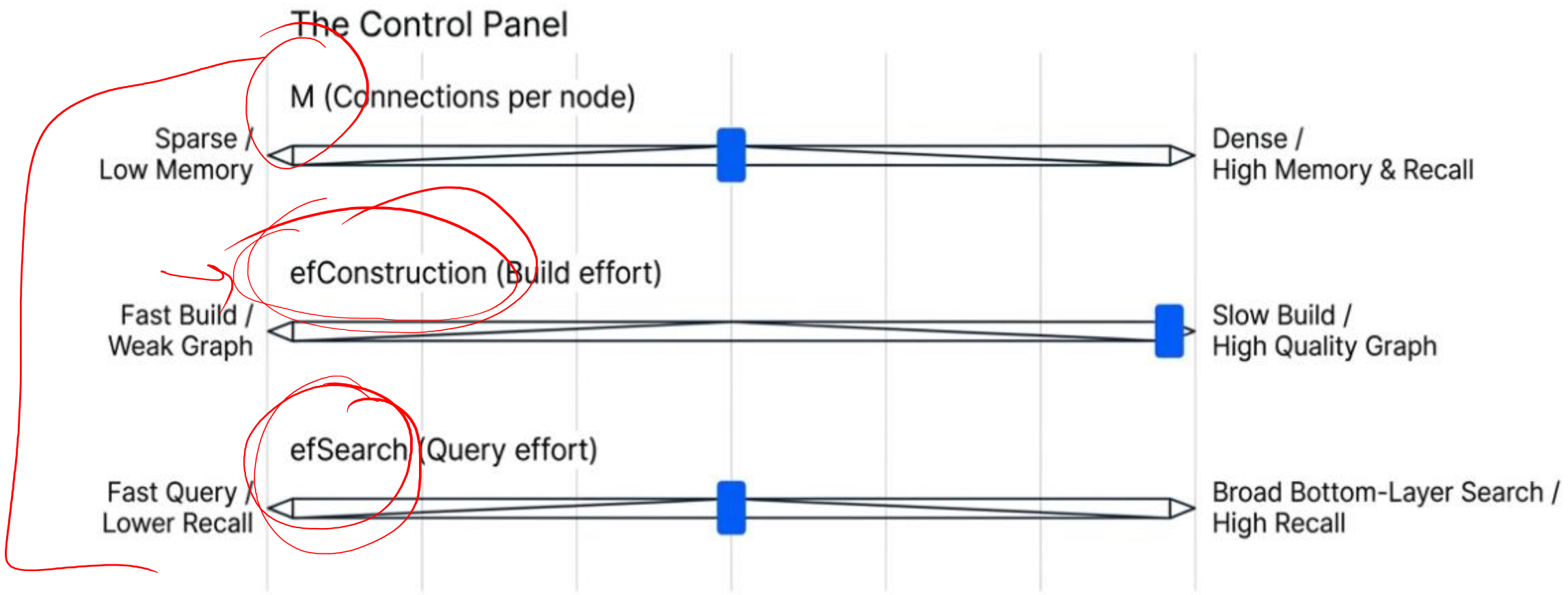
$m_L \gg 1$



~ 1

Dialing in performance and precision

HNSW provides a highly tunable recall-speed trade-off. M shapes the physical richness of the graph. efConstruction shapes graph quality during index building. efSearch determines the width of exploration at the bottom layer during a query.



Anatomy of Vector Database

Embedding Model

Creates vectors from raw data.

Vector Index

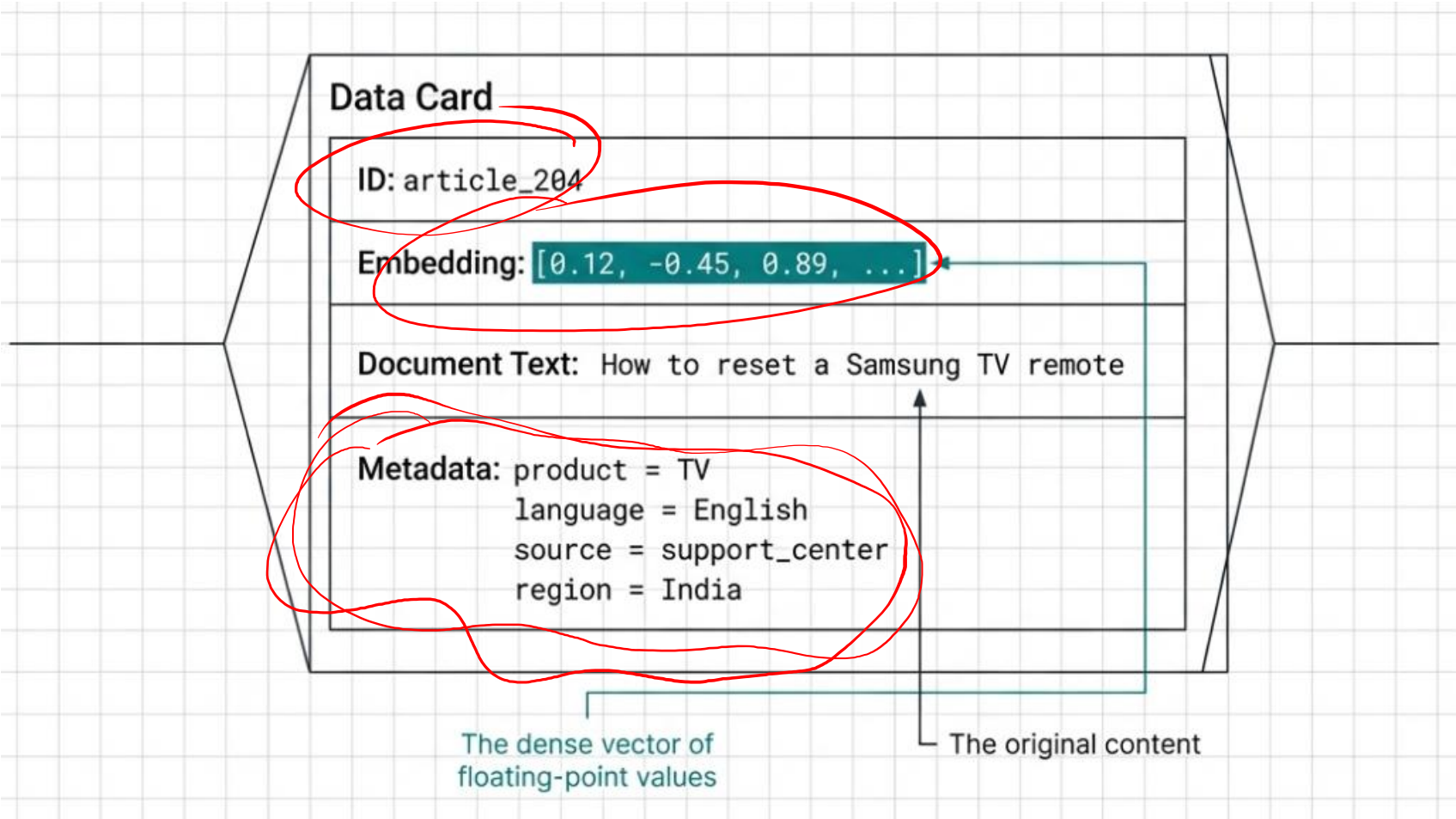
Organizes vectors to make nearest-neighbor search efficient (e.g., FAISS).

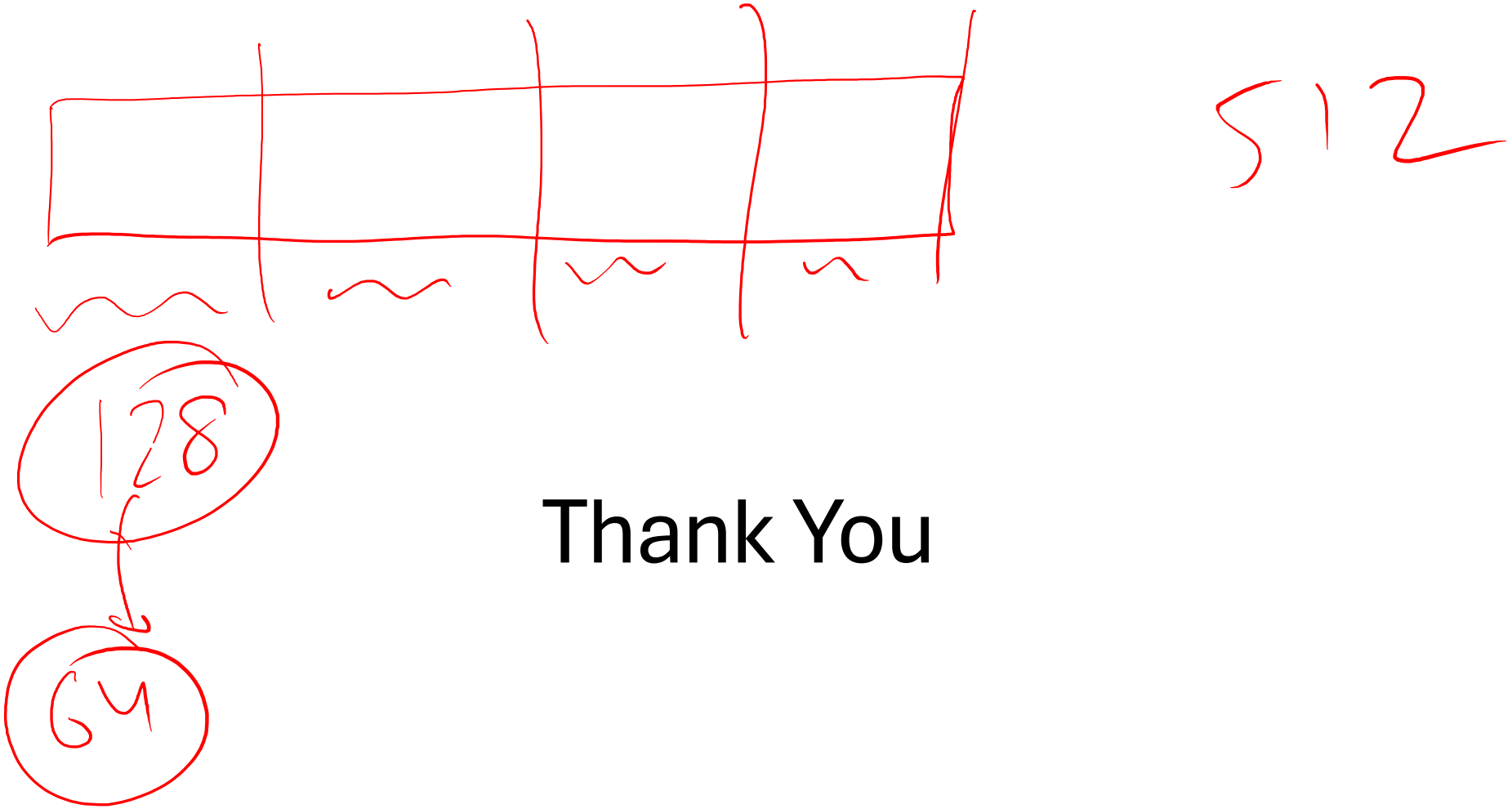
Vector Database

Stores the vectors, original content, and metadata, serving the APIs (e.g., Chroma).

A database stores the embeddings, but the index is what makes them searchable.

Vector Record





Thank You

Appendix